0KINGDOM OF SAUDI ARABIA |JAZAN UNIVERSITY
COLLEGE OF ENGINEERING & COMPUTER SCIENCE

Schedule Generator

Course with code: Artificial Intelligence (COMP 441)
Instructor: Dr.Anne Anoop

Name student	Student ID
Afnan Ogdi	202307421
Reem Badri	202305024
Aryaf Elaqi	202306972
Arwa Qumayri	202306439

Introduction

Students face problems when making schedules.
Classes overlap and it takes time to fix.
Our project builds an intelligent Schedule Generator.
It uses AI to create conflict-free timetables.
It can also keep one day free if the student wants.

Problem Definition

Problems students face:

- Many groups for each course.
- Time conflicts between groups.
- Manual trial-and-error is slow.
- Some students want a free day.

We model this as a Constraint Satisfaction Problem (CSP).

Algorithm Explanation

We use Backtracking Search Algorithm.

Steps:

1. Pick a subject.
2. Try each group.
3. Check constraints (time, day, free day).
4. If conflict → backtrack.
5. If valid → continue.

Result: All conflict-free schedules are generated.

Tools & Libraries

Tools we used:

- Google Colab
- Python
- Pandas (data handling)
- Datetime (time comparison)
- CSV files (course data)

Implementation Steps

1. Upload CSV file with course sections.
2. Enter course codes.
3. Select optional free day.
4. Program loads all options.
5. Backtracking generates valid schedules.
6. Results are printed.

Results & Screenshots

Importing Required Libraries

[ ]

```
import pandas as pd
from datetime import datetime
from google.colab import files
```

Parsing Time Strings

[1]
0s

```
def parse_time(t):
    t = t.strip()
    try:
        return datetime.strptime(t, "%H:%M").time()
    except ValueError:
        return datetime.strptime(t, "%H:%M:%S").time()
```

Checking if Two Time Intervals Overlap

[2]
0s

```
def times_overlap(a_from, a_to, b_from, b_to):
    s1, e1 = parse_time(a_from), parse_time(a_to)
    s2, e2 = parse_time(b_from), parse_time(b_to)
    return (s1 < e2) and (s2 < e1)
```

Results & Screenshots

Generating Non-Conflicting Schedules

[5] 0 s

```python
def generate_schedules(df, subjects, off_day=None, limit=10):
    choices = [] #Preparing Lecture Options
    for subj in subjects:
        options = df[
            (df['subject'].str.strip().str.lower() == subj.strip().lower()) |
            (df['subject_name'].str.strip().str.lower() == subj.strip().lower())
        ]
        groups = options.groupby('group')
        group_list = []
        for g, rows in groups:
            group_list.append(rows.to_dict('records'))
        if off_day:
            group_no_off = [g for g in group_list if all(r['day'].strip().lower() != off_day.strip().lower() for r in g)]
            choices.append({'subject': subj, 'no_off': group_no_off, 'all': group_list})
        else:
            choices.append({'subject': subj, 'all': group_list})
    schedules = []
    #Backtracking Algorithm to Build Schedules
    def backtrack(i, current, use_no_off):
        nonlocal schedules
        if len(schedules) >= limit:
            return
        if i == len(choices):
            schedules.append([r for group in current for r in group])  # flatten
            return
        entry = choices[i]
        options = entry['no_off'] if (off_day and use_no_off) else entry['all']
        for group in options:
            conflict = False
            for g in group:
                for c in current:
                    for c_r in c:
                        if c_r['day'].lower() == g['day'].lower() and times_overlap(
                            c_r['time_from'], c_r['time_until'], g['time_from'], g['time_until']):
                            conflict = True
                            break
                    if conflict:
                        break
                if conflict:
                    break
            if not conflict:
                current.append(group)
                backtrack(i + 1, current, use_no_off)
                current.pop()
    #Handling Optional Day Off
    if off_day:
        backtrack(0, [], True)
        if schedules:
            return schedules, True
        schedules = []
        backtrack(0, [], False)
        return schedules, False
    else:
        backtrack(0, [], False)
        return schedules, False
```

User Interface for File Upload

[8] 1 min

```python
print("Please upload your schedule CSV file:")
uploaded = files.upload()
file_path = list(uploaded.keys())[0]

df = pd.read_csv(file_path)
print("\nFile loaded successfully.")
print(df.head(), "\n")
```

```
Please upload your schedule CSV file:
اختيار ملفات schedule.csv
schedule.csv(text/csv) - 6066 bytes, last modified: 11100% - 2025/11/ done
Saving schedule.csv to schedule.csv

File loaded successfully.
   group subject            subject_name  day time_from time_until
0   2172   CS417  Application Devlopment  thu  13:00:00   13:50:00
1   2172   CS417  Application Devlopment  tue  14:00:00   14:50:00
2   2172   CS417  Application Devlopment  wed  14:00:00   15:50:00
3   2186   CS417  Application Devlopment  sun  14:00:00   14:50:00
4   2186   CS417  Application Devlopment  mon  15:00:00   15:50:00
```

User Input for Subjects and Day Off

[9] 20 s

```python
user_input = input("Enter the subjects or course codes (comma separated):\n> ").strip()
wanted_subjects = [x.strip() for x in user_input.split(",") if x.strip()]

off_day = input("\nDo you want a day off? (like: sun) or leave empty if not:\n> ").strip()
off_day = off_day if off_day else None
```

```
Enter the subjects or course codes (comma separated):
> CS417,CS4343,CS452,CS4533,CS4413

Do you want a day off? (like: sun) or leave empty if not:
> sun
```

Results & Screenshots

Generate Schedules and Display Results

[10]
✓ 0 s

```python
print("\nGenerating all possible non-conflicting schedules...")
schedules, off_ok = generate_schedules(df, wanted_subjects, off_day=off_day)

if not schedules:
    print("No possible schedule found for these subjects.")
else:
    if off_day:
        if off_ok:
            print(f"\nFound {len(schedules)} schedules with {off_day} OFF.")
        else:
            print(f"\nNo schedules with {off_day} off, but found {len(schedules)} possible schedules.")
    else:
        print(f"\nFound {len(schedules)} possible schedules.")
```

```
Generating all possible non-conflicting schedules...

Found 10 schedules with sun OFF.
```

Display Top Schedules

[11]
✓ 0 s

```python
for i, sched in enumerate(schedules[:5], 1):
    print(f"\nSchedule #{i}")
    for row in sched:
        print(f"{row['subject']} - {row['subject_name']} | "
              f"{row['day']} {row['time_from']} - {row['time_until']} (Group {row['group']})")
```

```
Schedule #1
CS417 - Application Devlopment | thu 13:00:00 - 13:50:00 (Group 2172)
CS417 - Application Devlopment | tue 14:00:00 - 14:50:00 (Group 2172)
CS417 - Application Devlopment | wed 14:00:00 - 15:50:00 (Group 2172)
CS4343 - Distributed copmuting | mon 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | wed 17:00:00 - 17:50:00 (Group 2169)
CS4343 - Distributed copmuting | tue 17:00:00 - 18:50:00 (Group 2169)
CS452 - Cloud Computing | mon 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | thu 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | tue 12:00:00 - 13:50:00 (Group 420)
CS4533 - Data Science | mon 12:00:00 - 12:50:00 (Group 2175)
CS4533 - Data Science | thu 10:00:00 - 10:50:00 (Group 2175)
CS4533 - Data Science | tue 08:00:00 - 09:50:00 (Group 2175)
CS4413 - Artificial Intelligence | mon 09:00:00 - 09:50:00 (Group 2322)
CS4413 - Artificial Intelligence | thu 09:00:00 - 09:50:00 (Group 2322)
CS4413 - Artificial Intelligence | thu 14:00:00 - 15:50:00 (Group 2322)

Schedule #2
CS417 - Application Devlopment | thu 13:00:00 - 13:50:00 (Group 2172)
CS417 - Application Devlopment | tue 14:00:00 - 14:50:00 (Group 2172)
CS417 - Application Devlopment | wed 14:00:00 - 15:50:00 (Group 2172)
CS4343 - Distributed copmuting | mon 18:00:00 - 18:50:00 (Group 2198)
CS4343 - Distributed copmuting | tue 18:00:00 - 18:50:00 (Group 2198)
CS4343 - Distributed copmuting | thu 18:00:00 - 19:50:00 (Group 2198)
CS452 - Cloud Computing | mon 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | thu 08:00:00 - 08:50:00 (Group 420)
CS452 - Cloud Computing | tue 12:00:00 - 13:50:00 (Group 420)
CS4533 - Data Science | mon 12:00:00 - 12:50:00 (Group 2175)
CS4533 - Data Science | thu 10:00:00 - 10:50:00 (Group 2175)
CS4533 - Data Science | tue 08:00:00 - 09:50:00 (Group 2175)
CS4413 - Artificial Intelligence | mon 09:00:00 - 09:50:00 (Group 2322)
CS4413 - Artificial Intelligence | thu 09:00:00 - 09:50:00 (Group 2322)
CS4413 - Artificial Intelligence | thu 14:00:00 - 15:50:00 (Group 2322)
```

Discussion & Analysis

Backtracking is efficient for scheduling.

It avoids invalid schedules early.

The project shows AI concepts:

- Search algorithms
- Constraint satisfaction
- State-space reasoning
- Decision pruning

Flexible: supports free day preference.

Conclusion

We achieved our goal:

- Generated conflict-free schedules.
- Applied AI search principles.

Future improvements:

- GUI interface
- Genetic Algorithms for best schedule
- Support for complex formats
- Automatic best schedule suggestion

References

- Python Documentation
- Pandas Library
- Google Colab

Thank you for your time ✨ ✨